程序设计综合实验

第 1 周：引言

黄韬
taohonker@gmail.com
昆明理工大学·系统科学系

2026-09-16

课前

讲师介绍

- 授课教师：黄韬（讲师）
- 学术背景：理学博士，兰州大学，理论物理专业
- 研究方向：统计与计算物理、数据科学，**欢迎想做科研的同学课后找我交流**
- 本课程的教学理念： 兼顾基础与前沿、 注重理论与实践结合、 鼓励主动探索与批判性思维
- 联系方式：QQ 课程群

课程安排

- 理论课：周四，3-5 节，多媒体 322
- 实验课：周四，1-2 节，理 108、110
- 助教：肖竹飞（理 418-2）
- 课程网站：https://git2.taohonker.space
- 作业与考核
  - 课堂参与与表现：10%，旷课一次扣 5 分，三次以上不予评分
  - 平时作业：20%，每次作业满分 100 分，按比例计入总评。作业评分为 A/B/C/D 四档，换算为 95/85/75/65 分，要求详见右侧
  - 上机考试：30%，在期末考试前利用一次**实验课**进行，闭卷、断网，只有一次机会，考试时间 120 分钟
  - 期末考试：40%，在考试周进行

笔记 1：作业要求

- 每次课上布置，无额外说明时截止时间为下周三晚 23:59。
- 作业形式多样，包括编程题、理论题、AI 探究题等。**手写作业需拍照上传**，不要丢弃，期末统一收取。
- 编程作业如指定语言，请务必使用指定语言完成。如未指定，可任选 C/C++、Julia 两类语言之一；使用其它语言导致评分出错的责任自负。
- 编程作业请务必保证代码**可运行**，且输出结果正确，代码风格良好，**有注释**。可使用 Markdown 文件或 Jupyter Notebook 等格式提交展示运行结果，但请**务必包含源代码文件**（可以是 `.c`, `.cpp`, `.jl`, `.ipynb` 格式）。非必要**请勿附加数据文件与图片**，控制作业体积。
- 作业提交：使用课程 Git 平台（https://git2.taohonker.space）提交。每次作业独立 fork 教师仓库（不改名），本地开发保留完整 Git 记录，导出为独立作业仓库，推送前必须先 pull 同步自己的 fork，再 push。详细规则见课程网站的作业提交说明。提交时有冲突、未按格式提交、未在截止时间前提交均评为 D。未提交不予评分。
- 不要完全依赖 AI 工具，**上机考试时会断网，没有重考机会！**

参考资料

课程讲义尽可能做到自足，有问题请及时提问、反馈，找到错误会有奖励。

参考资料：

- 互动精致开源教材：靳宇栋，Hello, 算法！
- 国内新近高质量教材：俞勇 等，《数据结构》（101 计划）
- 国内优秀数据结构教材：邓俊辉，《数据结构（C++语言版・第 3 版）》[1]
- 国外优秀算法入门：A. Levitin, 《算法设计与分析基础》
- 国外优秀数据结构与算法：D. Reed, J. Zelle, 《数据结构和算法（Python 和 C++ 语言描述）》
- 经典算法教材：T. Cormen 等, 《算法导论（原书第 3 版）》[2]

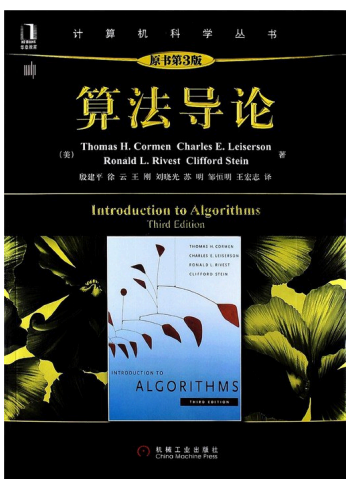












[1]清华大学，邓老师的书与授课材料包括视频均由作者本人在其主页免费提供，强烈推荐！

[2]算法圣经，内容全面，适合有一定基础的同学深入学习。非常难！英文版已出版 4th edition, 中文版译自 3rd edition.

引言

导论：为什么这门课如此重要?

导论：为什么这门课如此重要?

- 数据结构与算法是计算机科学的核心，是所有高效程序的基础。
- 在数据科学、人工智能、软件开发等领域，数据结构与算法决定了数据处理的效率和系统的可扩展性。
- 本课程不仅关注算法本身，更关注数据在计算机中的生命周期与价值转化，帮助大家建立“从内存到洞察到高效分析与算法求解”的整体视角。

课程总体介绍

- 本课程将系统讲解常用数据结构、典型算法和智能算法等内容。
- 课程内容贯穿 C、Python、Julia 三种主流语言，帮助大家理解底层实现与高层抽象的联系与区别。
- 强调数据结构与算法在数据生命周期各环节的作用。
- 鼓励大家利用 AI 工具（如 Copilot、ChatGPT 等）辅助学习，提升问题解决能力。

课程内容

- 简单数据结构与算法分析基础
  - ☒ 函数库与函数接口
  - ☒ 数组
  - ☒ Fibonacci 数列与递归
  - ☒ 冒泡排序
  - ☒ 插入排序
  - ☒ 快速排序
  - ☒ 分治
  - ☒ 归并排序
  - ☒ 单、双链表
  - ☒ 抽象数据类型
  - ☒ 链表上的排序
- 复杂数据结构、树与图
  - ☒ 栈与队列
  - ☒ 二叉树的数组表示
  - ☒ 树的遍历
  - ☒ 树的转动
  - ☒ AVL 树
  - ☒ 桶排序
  - ☒ 散列表与散列函数
  - ☒ 并行算法
  - ☒ 图的表示
  - ☒ 图的遍历
  - ☒ 无向连通图上节点的最短路径
  - ☒ KD 树
- 算法设计与智能算法
  - ☒ 算法的时间与空间效率
  - ☒ 神经网络算法与深度学习
  - ☒ 动态规划与优化算法
  - ☒ 旅行商问题
  - ☒ 贪心算法
  - ☒ 模拟退火算法
  - ☒ 元启发算法简介
  - ☒ 遗传算法
  - ☒ 蚁群算法
  - ☐ 【设计模式与算法设计基础方法】

- ☐ 【红黑树】
- ☐ 【基数排序】

案例一：校园路径规划



- 场景设定：假设你要从朗溪街 10 号门走到图书馆，如何选择最优路线？
- 用校园地图抽象为“图”结构，建筑物为节点，道路为边，距离为权重。
- 最短路径问题：如何用最少的时间或距离到达目的地？
- 延伸思考：
  - 如果路径的“代价”会随时间、天气、拥堵等因素动态变化，如何建模？
  - 如果考虑不同交通方式（步行、自行车、电动车），如何调整路径规划算法？
  - 如果体力有限，如何规划最优路径？
- 现实意义：路径规划算法广泛应用于地图导航、物流调度、网络路由等领域。

案例二：人际相似度度量

背景介绍：在现代社会，推荐系统无处不在，电商平台为你推荐商品，社交网络为你推荐好友，招聘网站为你推荐职位。这些系统的核心问题之一，就是如何衡量“人与人之间的相似度”。 - **推荐系统背后的本质问题：如何定义两个人之间的“距离”？距离越近，推荐的相关性越高。**

- 最初的思路：可以用空间位置（如地理坐标）来衡量距离。例如，住得近的同学可能更容易成为朋友。
- 进一步：籍贯、性别等基本属性也会影响相似度。比如同乡、同性别的人可能有更多共同话题。
- 再进一步：口味（如喜欢的食物、音乐）、社会单位归属（同班、同系、同社团）、兴趣爱好等，都是多维度的特征。
- 更复杂的情况：政治信念、价值观等深层次特征，也会影响人与人之间的距离，但这些特征往往难以量化。

案例二：人际相似度度量

- 问题的复杂性：每增加一个**特征维度**，人与人之间的“距离”就变成了一个**高维空间中的距离计算**问题。
- 不同类型的数据（数值型、分类型、集合型、文本型等）需要不同的数据结构来存储和处理。
- 需要**设计合适的“距离函数”**或“相似度函数”，如欧氏距离、曼哈顿距离、余弦相似度、Jaccard 相似系数等，根据实际需求灵活选择和组合。
- 数据结构的作用：用结构体（如 C 语言的 struct、Python 的 class、Julia 的 struct）来组织和存储每个人的多维特征。
- 构建函数和成员函数：通过编写函数（如成员方法）来自动计算两个人的距离，实现灵活、可扩展的相似度度量。
- 现实意义：这种多维度、可扩展的相似度计算方法，是现代推荐系统、社交网络分析、聚类分析等领域的基础工具。
- 小结：本例展示了现实问题的复杂性和数据结构、算法设计的必要性。只有通过合理的数据组织和高效的算法，才能在大规模数据中实现个性化推荐和智能分析。

课程核心观点

- 现实问题往往复杂多变，解决问题的能力，根本在于对数据的高效组织（数据结构）和高效处理（算法）。
- 数据结构决定了数据的存储方式和访问效率，算法决定了数据处理的速度和资源消耗。
- 数据结构与算法是数据科学、人工智能、工程开发等领域的“发动机”，直接影响系统的性能与可扩展性。
- 掌握数据结构与算法，不仅能提升编程能力，更能培养抽象建模和系统思考的能力。

小结

- 本节通过校园路径规划和人际相似度度量两个案例，帮助大家体会数据结构与算法在现实中的实际价值。
- 希望大家带着“如何用数据结构和算法解决现实问题”的思考，开启本学期的学习之旅。
- 鼓励大家在学习过程中积极提问、主动探索，并善用 AI 工具作为学习伙伴，不断深化理解与应用。

基础回顾与对比：C, Python, Julia 的内存观

基础回顾与对比：C, Python, Julia 的内存观

> 关注数据结构的内存占用与访问效率，理解底层实现与高层抽象的联系与区别，是学习数据结构与算法的基础。对于指针与引用的运用，贯穿了本学期课程的大部分内容，如果还不熟悉或已经遗忘，请务必认真复习本节内容。

C 语言的基石：直接内存抽象

- **指针：**
  - 指针是 C 语言的灵魂。每个变量在内存中都有一个地址，指针就是用来存储这个地址的变量。
  - 通过*（解引用）和 &（取地址）操作符，程序员可以直接访问和操作内存中的数据。
- **数组：**
  - 数组在 C 中是一块连续的内存区域，元素类型相同，便于高效访问和遍历。
  - 数组名本质上是指向首元素的指针，但数组和指针并不完全等价（如 sizeof、内存分配等）。
- **结构体：**
  - 结构体（struct）允许将不同类型的数据聚合在一起，形成自定义的数据类型。
  - 结构体的每个成员在内存中有固定的偏移量，便于整体管理和操作。
- **核心思想：**
  - C 语言让程序员直接管理内存，带来极高的性能和灵活性，但也要求开发者对内存分配、释放、越界等问题高度负责。

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
struct Student {
  int id;
  char *name;
  float score;
};
void print_student(struct Student *s) {
  printf("学号: %d, 姓名: %s, 分数: %.2f\n",
         s->id, s->name, s->score);
}
int main() {
  struct Student *stu = (struct Student
*)malloc(sizeof(struct Student));
  stu->id = 202501;
  stu->name = (char *)malloc(20 * sizeof(char));
  strcpy(stu->name, "张三");
  stu->score = 95.5;
  print_student(stu);
  free(stu->name);
  free(stu);
  return 0;
```

- 内存泄漏、野指针、缓冲区溢出等问题在 C 中非常常见，需格外小心。

```
23 }
```

Julia 的平衡：性能与抽象的双赢

- **类型系统：**
  - Julia 支持强类型和类型注解，允许开发者为变量、函数参数、返回值指定类型。
  - 类型注解有助于 JIT（即时编译）优化，提高代码运行效率。
- **数组：**
  - Julia 的数组是可变的、类型化的、连续的内存块，底层实现与 C 类似，支持高效的数值计算。
  - 支持多维数组，且索引从 1 开始。
- **结构体：**
  - `struct` 定义不可变类型，`mutable struct` 定义可变类型。
  - 不可变类型适合做数据快照、哈希表键等；可变类型适合需要频繁修改的场景。
- **内存与性能：**
  - Julia 通过类型推断和 JIT 编译，实现了接近 C 的性能，同时保留了高级语言的易用性和灵活性。
  - 合理使用类型注解和数据结构，可以极大提升代码效率。

```julia
# 不可变对象
a = 10; b = a; a = 20;
println(a, " ", b)  # 输出 20 10

# 可变对象
arr1 = [1, 2, 3]; arr2 = arr1; push!(arr1, 4);
println(arr2)  # 输出 [1, 2, 3, 4]

# 结构体与方法
mutable struct Student
  id::Int
  name::String
  score::Float64
end

function print_student(s::Student)
  println("学号: $(s.id), 姓名: $(s.name),
          分数: $(s.score)")
end

stu = Student(202501, "王五", 92.0)
print_student(stu)
```

引用与指针的关联与区别

- **相同点：**
  - 都是“间接访问”数据的机制，不直接存储数据本身，而是指向（或引用）数据在内存中的位置。
  - 允许多个变量指向同一份数据，实现数据共享。
- **区别：**
  - **C 语言的指针**是内存地址的直接操作，**支持算术运算（如 p+1），可以指向任意类型的数据**，甚至可以指向指针本身（多级指针）。
  - **引用（Reference）**（如 Python、Julia、Java 等）是更高级的抽象，通常不允许直接操作内存地址，也不支持指针算术。引用本身不可更改为指向其他对象（如 C++引用），但 Python/Julia 的变量可以重新绑定。
  - 指针需要手动管理内存（分配、释放），引用通常由语言自动管理内存（垃圾回收）。
- **Python**
  - 对于不可变对象（如 int），变量重新赋值会指向新对象，原对象不变。
  - 对于可变对象（如 list），多个变量指向同一对象，修改会影响所有引用。
- **Julia**[1]
  - Julia 变量本质上也是“绑定”到对象的引用，和 Python 类似。
  - 不可变对象赋值不会影响原对象；可变对象的修改会影响所有引用。

AI 互动环节

- 请大家打开自己熟悉的 AI 助手（如 ChatGPT、Copilot、DeepSeek 等）。

[1] Julia 也有对指针的支持，但在日常编程中很少直接使用。

- 提问任务：“请向 AI 提问：Python 中的变量到底是值传递还是引用传递？请用可变和不可变对象为例详细解释。”

实践技能：代码计时

为何要计时?

- 算法和程序的效率直接影响实际应用的可行性和用户体验。
- 直观的“快/慢”感受往往不准确，科学计时是衡量和优化算法性能的第一步。
- 通过计时，可以比较不同算法、不同实现方式的优劣，指导后续优化。

C 语言：使用 clock() 计时

- C 语言提供 <time.h> 头文件中的 clock() 函数，用于测量程序运行的 CPU 时间。
- 基本用法：在代码片段前后分别调用 clock()，计算差值并除以 CLOCKS_PER_SEC 得到秒数。

```c
#include <stdio.h>
#include <time.h>

int main() {
    clock_t start, end;
    double cpu_time_used;
    start = clock();

    // 需要计时的代码
    for (int i = 0; i < 1000000; i++);

    end = clock();
    cpu_time_used = ((double) (end - start)) / CLOCKS_PER_SEC;
    printf("运行时间: %f 秒\n", cpu_time_used);
    return 0;
}
```

- 注意：clock() 计的是 CPU 时间，受操作系统调度影响较小，适合计时纯计算型代码。

C++：使用 chrono 库计时

- C++11 及以上推荐使用 <chrono> 标准库进行高精度计时，语法简洁，跨平台，适合微秒级别的性能分析。

```cpp
#include <iostream>
#include <chrono>

int main() {
    // 获取起始时间点
    auto start = std::chrono::high_resolution_clock::now();

    // 需要计时的代码
    long long sum = 0;
    for (int i = 0; i < 1000000; ++i) sum += i;

    // 获取结束时间点
    auto end = std::chrono::high_resolution_clock::now();

    // 计算耗时（单位：微秒）
    auto duration = std::chrono::duration_cast<std::chrono::microseconds>(end - start);
    std::cout << "运行时间: " << duration.count() << " 微秒" << std::endl;
    return 0;
}
```

- chrono 支持多种时间单位（秒、毫秒、微秒、纳秒等），可根据需求灵活选择。

- 推荐用于 C++性能分析和竞赛编程等场景。

Julia：`@time` 与 `@btime` 的对比

- Julia 内置 `@time` 宏，可直接计时并显示内存分配信息。
- 推荐使用 `BenchmarkTools.jl` 包的 `@btime`，更专业、结果更稳定。

```
# @time 示例
A = [1, 2, 3]
@time sum(A)

# @btime 示例（需先安装 BenchmarkTools）
using BenchmarkTools
@btime sum($A)
```

- `@time` 只运行一次，结果易受 JIT 编译、全局变量等影响。
- `@btime` 会多次运行、自动插入 `$` 避免全局变量干扰，并报告最优耗时和内存分配。

关键总结

- 计时工具的选择要结合语言特性和实际需求。
- C 语言需手动计时，适合底层性能分析。
- Python 的 `timeit` 和 Julia 的 `@btime` 能自动多次测试，结果更可靠。
- Julia 的 `@btime` 还能显示内存分配，有助于发现隐藏的性能瓶颈。
- 养成科学计时和性能分析的习惯，是成为高效程序员的必备技能。

> 更深入的运行时性能分析，需要借助专业的性能分析工具（Profiler），如：`gprof`、`valgrind`、`perf` 等。

实践：为 Fibonacci 递归与迭代实现计时

阅读 C/Python/Julia 版本的 Fibonacci 递归与迭代实现代码，为两种实现分别计时，比较性能差异。

宏观视角：数据的完整生命周期

数据科学的内涵与外延

- **定义：** 数据科学是研究如何从数据中提取知识和洞察的跨学科领域，融合了统计学、计算机科学和具体应用领域的知识。
- **三元论：**
  - **统计学/数学**：为数据分析提供理论基础和建模工具。
  - **计算机科学**：提供数据存储、处理、算法实现和系统架构的能力。
  - **领域知识**：理解数据的实际背景，提出有意义的问题，解释分析结果。
- **现实意义：** 数据科学不仅仅是编程或建模，更是贯穿数据全生命周期的系统性工程。

数据的完整生命周期

- **获取 (Acquisition)：**
  - 数据的来源多样，包括传感器、网络日志、问卷调查、第三方 API 等。
  - 需要考虑数据采集的自动化、实时性、准确性和合规性。
  - 例：爬虫抓取网页数据、IoT 设备实时上传传感数据。
- **清洗与预处理 (Cleaning & Preprocessing)：**
  - 原始数据往往存在缺失、异常、重复、格式不一致等问题。
  - 典型操作包括去重、填补缺失值、异常检测、数据类型转换、特征工程等。
  - 这是数据分析中最耗时、最基础的环节。
- **存储与管理 (Storage & Management)：**
  - 数据量小可用本地文件（如 CSV、Excel），大数据需用数据库（如 MySQL、MongoDB、Hadoop 等）。
  - 需关注数据的安全性、可扩展性、访问效率和备份恢复。
  - 元数据（数据的数据）管理同样重要，便于数据追踪和治理。
- **流转 (Movement & Flow)：**
  - 数据在不同系统、平台、部门间的流动（如 ETL：Extract-Transform-Load）。
  - 包括批量处理和流式处理，涉及数据同步、迁移、实时推送等。
  - 例：企业数据仓库的构建、实时监控系统的数据流。

数据的完整生命周期

- **分析与建模 (Analysis & Modeling)：**
  - 包括统计分析、机器学习、深度学习等方法，挖掘数据中的模式和规律。
  - 需要选择合适的算法、特征和评估指标，结合领域知识解释结果。
  - 例：用户行为预测、图像识别、文本情感分析。
- **积累 (Accumulation) -> 数据资产 (Data Assets)：**
  - 数据经过长期积累和加工，形成可复用、可共享的“数据资产”。
  - 数据资产是企业和组织的核心竞争力之一。
  - 例：用户画像库、知识图谱、训练好的模型。
- **可视化与沟通 (Visualization & Communication)：**
  - 通过图表、仪表盘、交互式报告等方式，将复杂数据和分析结果直观展示给决策者和用户。
  - 强调数据故事讲述和跨部门沟通能力。
- **归档与销毁 (Archival & Destruction)：**
  - 对不再活跃但需保留的数据进行归档，降低存储成本。
  - 到期或敏感数据需安全销毁，确保合规和隐私保护。
- **贯穿始终的治理与元数据管理 (Governance & Metadata)：**
  - 包括数据质量控制、权限管理、合规审计、数据血缘追踪等。
  - 元数据管理帮助理解数据的来源、结构、用途和变更历史，是数据生命周期管理的基础。

数据结构与算法：生命周期的引擎

- **核心信息：**
  - 数据结构和算法在数据生命周期的每个环节都扮演着基础性角色。
  - 例如：
    - 数据获取阶段用队列/缓冲区管理实时流数据。
    - 清洗阶段用哈希表去重、树结构做高效查找。
    - 存储阶段用 B+树等索引结构提升数据库检索效率。
    - 分析与建模阶段用矩阵、图、堆等结构支撑复杂算法。
    - 可视化阶段用图结构、树状结构组织层级信息。
  - 没有高效的数据结构和算法，数据科学的每一步都可能成为瓶颈。

AI 互动环节

- **提问任务：** “请向 AI 提问：‘在数据生命周期的各个阶段，通常会用到哪些具体的数据结构和算法？请每个阶段列举 3 个例子并简要说明。’”
  - 对于看不懂的回答，可以继续追问，直到理解为止

附录：C 指针与动态内存复习

附录：C 指针与动态内存复习

本附录浓缩自 C 语言指针与动态内存两讲的核心内容，供同学们快速回顾。若对某一部分仍有疑问，请结合完整讲义与教材继续深入。

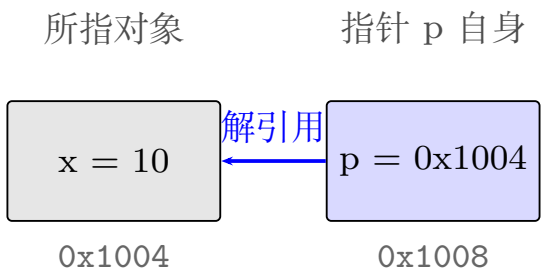
什么是指针

指针是一种特殊的变量：它所存储的数值不是“数据本身”，而是一个**内存地址**。

- 普通变量：保存数据本身。
- 指针变量：保存某个数据的地址。

指针具有**双重性**：它既是一个“值”（地址），也是对另一对象的“引用”。

```c
int x = 10;
int *p = &x;
printf("x 的值: %d\n", x);
printf("p 存储的地址: %p\n", (void*)p);
printf("通过 p 访问的值: %d\n", *p);
```



p 既是地址 0x1004
也是对 x 的引用

图 1: 指针的双重性：指针变量 p 存储地址 0x1004，通过解引用指向对象 x

- **指针 p 自身也占据一块内存**。
- 指针的“步长”由所指类型的大小决定：`p+1` 前进 `sizeof(*p)` 字节。
- 指针本身的大小由平台决定（64 位平台常为 8 字节），与所指类型无关。

lvalue 与 rvalue

- **lvalue（左值）**：表示一个**可寻址的内存位置**，有地址，可出现在赋值语句左边。
- **rvalue（右值）**：表示一个**具体的数值或临时结果**，没有固定地址，只能参与计算。

指针的双重性在此体现得尤为明显：

```
int x = 5;
int *p = &x;
*p = 20;            // *p 作为左值，修改 x
int y = *p + 1;     // *p 作为右值，提供 x 的值参与运算
```

> 如果 p 指向无效地址（未初始化或悬空），那么 *p 无论作为左值还是右值，都会导致程序崩溃。

指针的声明与初始化

声明语法

```
int *p;         // 指向 int 的指针
```

初始化方式

1. 指向某个变量：`int *p = &x;`
2. 置为 `NULL`：`int *p = NULL;`
3. 延迟初始化：先声明，后赋值。

小心 1：安全提醒

- **务必初始化指针**：未初始化的指针里存的是随机地址，解引用会导致程序崩溃。
- **`NULL` 是安全的默认值**：暂时没有目标时先赋为 `NULL`，使用前可检查。
- **不要随意解引用**：只有确认指针指向有效地址时，才能使用 *p。

取地址与解引用

指针的两个核心操作：

- **取地址 &**：得到某个变量在内存中的位置。
- **解引用 ***：通过指针访问或修改它所指向的内容。

```
int x = 10;
int *p = &x;   // &x 表示 x 的地址
*p = 20;       // 通过 p 修改了 x 的值
printf("%d\n", *p);  // 读取 x 的值
```

> 打印地址要使用 %p，并把指针强制转换为 void*：printf("%p\n", (void*)p);，这是定义良好且可移植的方式。

指针算术

指针不仅可以存储地址，还能进行算术运算：

- `p++`：指针向后移动一个元素。
- `p+n`：指针向后移动 `n` 个元素。
- `p1 - p2`：计算两个指针之间的距离（元素个数），结果类型为 `ptrdiff_t`。

指针算术的步长不是字节，而是**指针所指类型的大小**。

```
int arr[5] = {10, 20, 30, 40, 50};
int *p = arr;
for (int i = 0; i < 5; i++) {
    printf("%d\n", *(p+i));  // 等价于 arr[i]
}
```

合法性与边界

- `p1 - p2` 只有在两个指针指向同一个数组（或末端后一位）时才有定义。
- 指针算术**仅在同一数组内合法**，越界是未定义行为。
- 允许计算“末端后一位”（如 `arr + n`），常用于迭代结束判断，但**不能解引用它**。

```
1 int arr[5] = {10, 20, 30, 40, 50};
2 int *p = arr;
3 int *end = arr + 5;  // 末端后一位，合法但不可解引
  用
4 while (p != end) {
5     printf("%d\n", *p);
6     p++;
7 }
```

特殊指针类型

- **`void*`（泛型指针）**：可以指向任意类型的数据，常用于 `malloc` 返回值。不能直接解引用，必须先转换为具体类型指针。
- **`const` 修饰的指针**：
  - `const int *p;`：所指对象只读，指针可重新指向。
  - `int *const p;`：指针本身固定，所指对象可修改。
  - `const int *const p;`：指针与所指对象都不可修改。
- **`int**`（指向指针的指针）**：常见于多级间接访问，如二维数组、动态内存分配。

动态内存：为什么需要

栈区、堆区、数据区、代码区

- **栈区**：局部变量与调用帧，随作用域自动释放。
- **堆区**：运行时动态分配，由程序员用 `malloc/calloc/realloc/free` 管理。
- **数据区**：全局/静态变量（含 BSS 未初始化区）。
- **代码区**：只读指令区域。

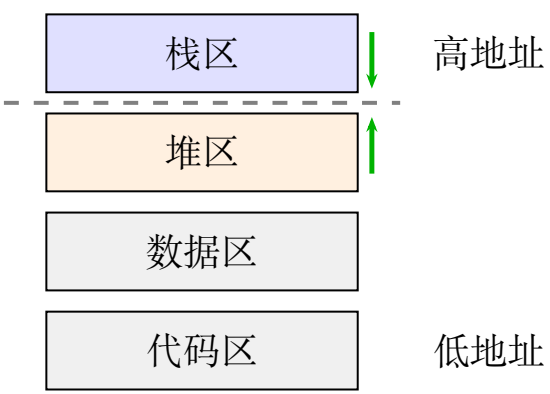


图 2: 进程内存布局：代码区、数据区、堆区、栈区，堆栈相向增长

为什么需要动态内存

- **数据规模不确定**：输入文件大小、网络包数量等只有运行时才知道。
- **生命周期跨越作用域**：函数返回后仍需存活的数据无法用栈变量安全承载。
- **大对象与灵活结构**：链表、树、图、可扩容容器需要分层分配与按需扩容。
- **资源统一管理**：显式 `free` 支持确定性的释放，避免长尾占用。

栈 vs 堆

- 栈：自动管理、容量有限（典型几 MB），适合小而短命的局部对象。
- 堆：手动管理、容量取决于进程可用内存，适合大对象与跨作用域存活的数据。

malloc(size)

- 向堆申请至多 size 字节的原始内存，返回首地址（void*）。
- 未初始化（脏数据），读出未初始化内容属于未定义行为。
- 失败返回 NULL。

calloc(n, elem_size)

- 为 n 个元素分配大小各为 elem_size，原内存**全部置零**。
- 适合需要默认值为 0、避免未初始化不确定性的场景。

```
1 int n; scanf("%d", &n);
2 int *arr = (int*)malloc((size_t)n * sizeof(int));
3 if (!arr) { fprintf(stderr, "分配失败\n"); return
  1; }
4 for (int i = 0; i < n; i++) arr[i] = i;
5 free(arr); arr = NULL;
6
7 int *zeros = (int*)calloc(8, sizeof(int));
8 if (!zeros) { fprintf(stderr, "calloc 失败\n");
  return 1; }
9 free(zeros);
```

realloc(ptr, new_size)

- 尝试将 ptr 指向的旧内存调整为 new_size。可能原地扩展（地址不变），也可能迁移到新块（地址改变，旧块被释放并复制数据）。
- 成功返回新指针地址，失败返回 NULL 且原内存不变。
- **安全模式**：用临时指针接收返回值成功再更新原指针。

```
int *tmp = (int*)realloc(arr, (size_t)new_size);
if (tmp) {
    arr = tmp;   // 统一以新值为准
} else {
    // 失败：arr 保持旧值，可继续使用或释放
}
```

free(ptr)

- 将 ptr 指向的动态内存归还给堆管理器。
- 释放后立刻置 NULL：free(p); p = NULL;，避免悬空与二次释放。
- 只释放动态分配的起始地址，不得对中间地址或成员指针调用 free。

提示 1：malloc vs calloc 选择指南

- 需要固定大小的未初始化块：选 `malloc`。
- 需要自动清零（安全默认值）：选 `calloc`。
- 已有数据按需调整容量：选 `realloc`，用临时指针接收返回值。
- 不再使用的内存：调用 `free` 并将指针置空。

动态数组与扩容策略

扩容策略采用**指数增长**（如容量翻倍），以降低复制次数并减少碎片化。

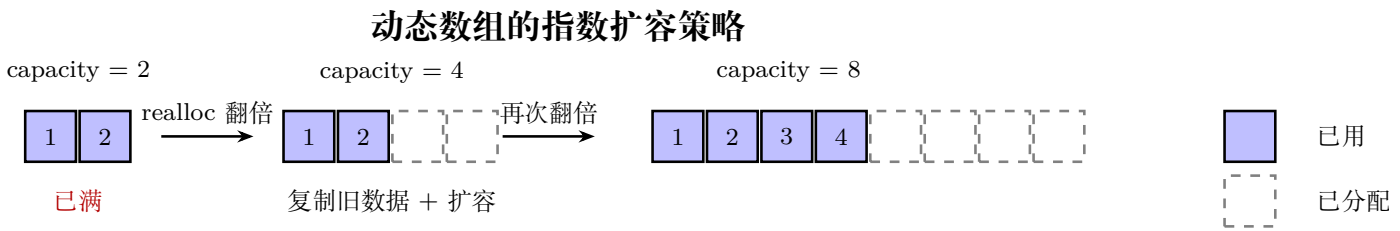


图 3: 动态数组的指数扩容：容量 2 到 4 到 8 逐次翻倍，均摊常数时间

```c
typedef struct { int *data; int size; int capacity; } DynamicArray;
static int ensure_capacity(DynamicArray *da, int need) {
    if (need ≤ da->capacity) return 1;
    int newcap = da->capacity ? da->capacity : 2;
    while (newcap < need) newcap *= 2;
    int *tmp = (int*)realloc(da->data, (size_t)newcap * sizeof(int));
    if (!tmp) return 0;   // 失败：保留旧内存与旧指针
    da->data = tmp; da->capacity = newcap; return 1;
}
static void push_back(DynamicArray *da, int v) {
    if (!ensure_capacity(da, da->size + 1)) fprintf(stderr, "realloc 失败\n"); exit(1);
    da->data[da->size++] = v;
}
```

安全与常见错误

警惕 1：指针与动态内存的常见陷阱

- **分配返回值未检查**：所有 malloc/calloc/realloc 必须判空，失败立即返回或进入统一清理路径。
- **未初始化使用**：malloc 后立即显式初始化，或改用 calloc。
- **悬空指针**：free 后继续读写、返回栈上地址。规避：free(p); p = NULL;，不返回局部对象地址。
- **内存泄漏**：丢失最后一个可达指针（被覆盖、异常路径、提前返回）。规避：分配附近就写出对应 free()。
- **二次释放**：统一所有权归属，仅“唯一所有者”负责释放，释放后立即置空。
- **越界访问**：封装容量与长度抽象（如 DynamicArray），所有写入前做边界检查。
- **realloc 误用**：直接 arr = realloc(arr, new_size) 会在失败时丢失原指针。总用临时指针接收返回值。

结构体指针

基本语法

```
1 struct Student {
2     int id;
3     char *name;
4     float score;
5 };
6
7 struct Student s;
8 struct Student *p = &s;
```

语法糖 →

```
p->id = 202501;         // 等价于 (*p).id = 202501;
p->score = 95.5;        // 等价于 (*p).score = 95.5;
```

- → 是 (*p).成员 的简写，避免复杂的括号嵌套，提高可读性。

为什么常用结构体指针而不是实例

- **函数参数传递**：直接传结构体实例会复制整个结构体，开销大；用指针传递只复制地址，效率高，还能让函数修改调用者的内容。
- **动态内存管理**：结构体可能很大或数量不确定，用指针在堆上动态分配，灵活管理生命周期。
- **数据结构设计**：链表、树、图等结构都依赖指针连接节点。

```
struct Node {
    int value;
    struct Node *next;
};
```

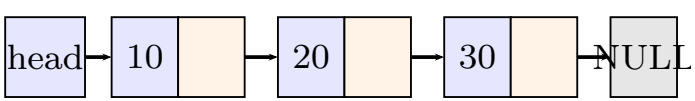


图 4: 单链表结构：每个节点含数据域与指针域，靠 next 串联，末尾指向 NULL

结构体的内存布局与对齐

结构体的内存布局可能包含**填充（padding）**，以满足对齐要求。

```
struct A {
    char c;    // 1 字节
    int x;     // 4 字节
};
```

- `char c` 占 1 字节，后面填充 3 字节，使 `int x` 按 4 字节对齐。
- 整个结构体大小为 8 字节，而不是 5 字节。

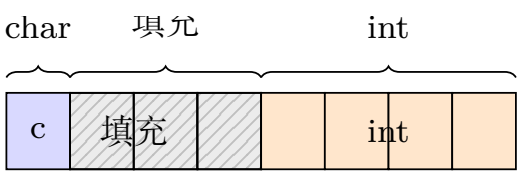


图 5: 结构体的内存对齐：char 占 1 字节后填充 3 字节，使 int 按 4 字节对齐，整体 8 字节

```c
#include <stdio.h>

struct A { char c1; char c2; int x; };
struct B { int x; char c; };

int main() {
    printf("sizeof(struct A) = %zu\n", sizeof(struct A));
    printf("sizeof(struct B) = %zu\n", sizeof(struct B));
    return 0;
}
```

可能输出：

```
sizeof(struct A) = 8
sizeof(struct B) = 8
```

> 即使成员顺序不同、个数不同，编译器也可能插入填充字节以保证对齐。需用 `sizeof` 验证实际大小。

结构体与动态内存结合

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

struct Student {
    int id;
    char *name;
    float score;
};

void print_student(struct Student *s) {
    printf("学号: %d, 姓名: %s, 分数: %.2f\n",
           s->id, s->name, s->score);
}

int main() {
    struct Student *stu = (struct Student *)malloc(sizeof(struct Student));
    if (!stu) { fprintf(stderr, "alloc failed\n"); return 1; }
    stu->id = 202501;
    stu->name = (char *)malloc(20 * sizeof(char));
    if (!stu->name) { free(stu); return 1; }
    strcpy(stu->name, "张三");
    stu->score = 95.5;
```

```
24      print_student(stu);
25      free(stu->name);
26      free(stu);
27      return 0;
28  }
```

重要 1：释放顺序
动态分配的结构体若含有动态分配的内部成员（如 char *name），必须**先释放内部成员，再释放结构体本身**，否则会造成内存泄漏。

本附录仅覆盖指针与动态内存的核心概念。void* 与通用内存、函数指针、多维数组分层分配、带标签联合（tagged union）等进阶内容，将在后续课程中继续展开。

作业

作业

1. 复习 C 语言指针、数组、结构体的概念，写一个简单的 C 程序，定义一个结构体表示学生信息（学号、姓名、成绩），并实现一个函数打印学生信息。
2. （可选）使用 Python 或 Julia 实现类似的学生信息类，并编写方法打印信息，体会两种语言在内存管理和引用传递上的差异。
3. 练习使用各语言的计时方法（至少两种语言），分别为 C、Python、Julia 实现的 Fibonacci 递归和迭代版本计时，比较性能差异。Fibonacci 序列的定义为：

$$F(0) = 0, \quad F(1) = 1, \quad F(n) = F(n-1) + F(n-2) \quad (n \geq 2).$$

4. 阅读并理解“链表”数据结构的基本概念，以及三种语言中链表的实现方式（参考提供的代码），准备下节课的学习。